ISTA 130

`for` Loops, String Concatenation and Repetition

What's new?

Review terminology

More variables

More `for` loops

NOTE: Make sure to put test_functions.py in the same folder as functions.py and run it with the python command.

> *If you're still having trouble with functions, please get help ASAP!*

Last time

More Turtle graphics

More functions

`for` loops

Types: what type are these?

➔ `45`	***int***
➔ `-12.7`	***float***
➔ `'45'`	***str*** or ***string***
➔ `'Freya'`	***str*** or ***string***
➔ `tallulah` or `az_turtle`	***turtle***

Work that Turtle

What do you think this will do?

```
def hmmm(a_turtle, side_len):
    for i in range(5):
        a_turtle.forward(side_len)
        a_turtle.right(144)
    return None
```

Indentation

Let's see what happens when we're sloppy with indentation.

Bugs in Your Code

It takes experience to get good at debugging.

➔ Introducing your own bugs on purpose is a good way to learn about error messages since you know what is actually wrong.

When you have a bug (something isn't working):

➔ Spend some time trying to figure it out and fix it.

➔ But don't spend hours - it will frustrate you and make you hate programming. Get help from us. You can email us your `.py` file if you can't come see us in office hours.

➔ Don't let bugs in your code become bugs in your brain.

Note on Exponents

- Standard math interpretation: $x^{y^z} \rightarrow x^{(y^z)}$

  $2^{3^2} \rightarrow 2^{(3^2)} \rightarrow 2^9 \rightarrow 512$

- Python interpretation: $x^{y^z} \rightarrow x^{(y^z)}$

  `2 ** 3 ** 2` $\rightarrow 512$

- Excel interpretation: $x^{y^z} \rightarrow (x^y)^z$

  `2 ^ 3 ^ 2` $\rightarrow$ `(2 ^ 3) ^ 2` $\rightarrow 8$ `^ 2` $\rightarrow 64$

  - to get that answer in Python:

    `(2 ** 3) ** 2` $\rightarrow 64$

- As always, if you're unsure about something use parentheses

Terminology

Variable: a name that refers to an object.

➔ `size = 200`

➔ `tallulah = turtle.Turtle()`

Parameter: variable used by a function to refer to argument values passed when the function is called.

➔ `def square(a_turtle, side_len):`

Argument: an object passed to a function. Can be a variable or a literal.

➔ `square(tallulah, 75)`

➔ `square(myrtle, 200)`

Scope

The region of code where a variable is visible (i.e., where Python knows what you're talking about).

- ➔ Parameters are only visible within their functions.
- ➔ Variables defined within functions are only visible to Python within those functions.
  - ◆ `main()` 's variables are visible within `main()` only.
  - ◆ `square()` 's variables are visible within `square()` only.

"Visible" means that Python knows what object that variable refers to.

Variables

In the for Loops lecture, we talked about how we could use a `size` variable for our house, so that we could just update one variable when we wanted to change the size (one line of code) vs updating all of the measurements wherever the "size" was needed.

Structuring our code this way would mean

This:

```
house(yertle, 200)
yertle.penup()
yertle.forward(250)
yertle.pendown()
house(yertle, 150)
...
```

became this:

```
size = 200
house(yertle, size)
yertle.penup()
yertle.forward(size * 1.25)
yertle.pendown()
house(yertle, size * 0.75)
...
```

Variables

In Python and many other programming languages, `=` is called the ***assignment operator***.

It is not the "`=`" you know from `2 + 2 = 4`

Read **`size = 200`** as "size gets assigned 200".

When using size afterwards, `y = size / 2`, say "`size` refers to `30`" or "`size` points at `30`".

Variables

You can't use a variable before assigning something to it.

```
>>> size
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'size' is not defined
```

What is the exception?

Variables

```
>>> x = 100    # x gets assigned 100
>>> x / 2      # x refers to 100
50.0
>>> x = 'hello'    # x gets assigned 'hello'
>>> x              # x refers to 'hello'
'hello'
```

The = assigns an object representing the value of the expression on the right to the variable.

I think of the variable as a basket and = dumps out whatever is already in the basket and puts in something new.

More terminology: Literal Values

A ***literal*** is an actual value (i.e., not a *variable*) as represented in the programming language.

➔ `999` `# an integer literal`
➔ `3.14159` `# a float literal`
➔ `'hello'` `# a string literal`

TL;DR

➔ **Variables refer to values.**
➔ **Literals are values.**

Variables

Let's do some assignments.

In the sequence of instructions to the right, changing y did not change x, even after we said x = y.

The assignment statement causes the name on the left of the = to refer to a different object.

The stuff on the right just determines which object.

```
>>> x = 100
>>> y = 50
>>> x = y
>>> x
50
>>> y = 99
>>> x
50
```

```
x = y + z
```

`x` gets assigned an object representing the current value of the sum of `y` and `z`.

The right-hand side of the assignment statement is evaluated only once.

```
>>> y = 50
>>> z = 1
>>> x = y + z
>>> x
51
>>> y = 99
>>> z = 88
>>> x
51
```

`for` Loop

Here is an example of using a `for` loop in Python:

```
for i in range(4):
    yertle.forward(side_length)
    yertle.left(90)
```

What's that `i`? => A **loop control variable**

What's inside it?

➔ To find out, we'll take a slight detour.

Detour: the `print` function

Type this code in a script and run the script, or type it into the Python shell.

```
print(100)
print('hi')
print(100, "hi")
print(100, "hi", 3.14, 'pi')
x = 5
print(x)
```

So `print` means "print to the terminal".

➔ Python evaluates each of the parameters and passes them to `print` which prints them to the terminal/command line, separating each with a space.

More on `print`

We can print any expression (or expressions separated by commas)

In the shell try these:

```
name = "Ziggy"
print(name)
print('Hello', name)
print('Hello', 'name')
```

Without the quotes, Python sees a variable called `name` to be evaluated.

With quotes, Python sees a string with 4 letters in it.

Back to Our `for` Loop

Let's try printing the contents of `i`:

```
for i in range(4):

    print(i)
```

```
>>> for i in range(4):
...     print(i)
...
0
1
2
3
```

Let's walk through it

```
>>> for i in range(4):
...     print(i)
...
0
1
2
3
```

1. Python creates a *variable* named `i`
    a. (you get to choose the variable's name)
2. Then, behind the scenes, Python does this: `i = 0`
    a. (programmers count from 0)
3. It then executes the indented statements in order
    a. replacing `i` with the value `i` currently refers to
        i. (e.g., a `0` the first time through the loop)
4. When it reaches the last indented statement it goes back to the top of the loop
    a. And behind the scenes Python does this: `i = 1`
    b. The value of `i` increases by 1
5. It continues until it has repeated the process 4 times

`for` Loops

```
def square(a_turtle, side_len):
    for i in range(4):
        a_turtle.forward(side_len)
        a_turtle.right(90)
    return None
```

Notice the indentation:

- ➔ The `for` loop is indented one level to put it inside the function.
- ➔ The turtle method calls are indented two levels to put them inside the `for` loop.
- ➔ `return None` is only indented once, because it's inside the function but not inside the `for` loop. Watch out for this!

How is this Useful?

The variable `i` successively takes on different values.
We can use this to do interesting things.

We could do this:

```
for i in range(36):
    square(yertle, 200)
    yertle.right(10)
```

What will this do?

```
for i in range(100):
    square(yertle, i)
    yertle.right(10)
```

How is this Useful?

```
for i in range(100):
    square(yertle, i)
    yertle.right(10)
```

What's happening?

- ➔ Remember what `i` was doing when we printed it.
- ➔ On each *iteration* of the `for` loop the variable `i` has a different value inside it
  - ◆ So `square(i)` does something different on each iteration!

Using the Loop Variable

What if we do this?

```
for i in range(100):
    square(yertle, i * 5)
    yertle.right(10)
```

➔ The *loop control variable* `i` is just a variable

- We can use it in expressions and statements (like any other variable)
- Just remember that before each *iteration* the `for` loop sets `i` to its next value.

Loop Control Variable

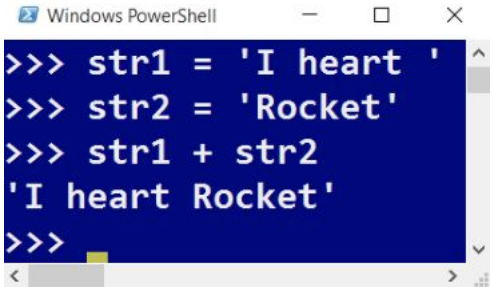
There's nothing special about the variable's name. This works too:

```
for master_control in range(100):
    square(myrtle, 5 * master_control)
    myrtle.right(10)
```

It's just a convention in many programming languages to use the letter `i` as the loop control variable's name if it is just counting.

String Concatenation and Repetition

We can add strings together using the plus sign (concatenation):

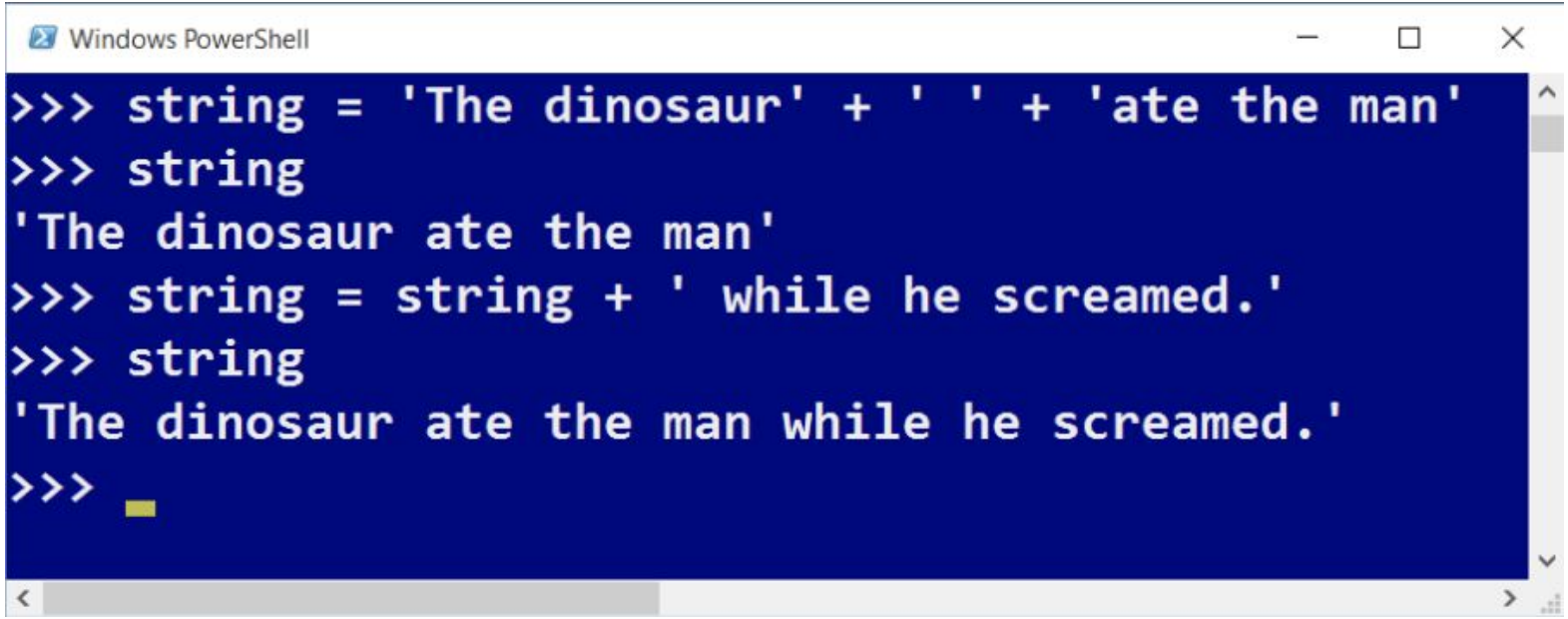


We can multiply a string by an integer with the multiplication (repetition):

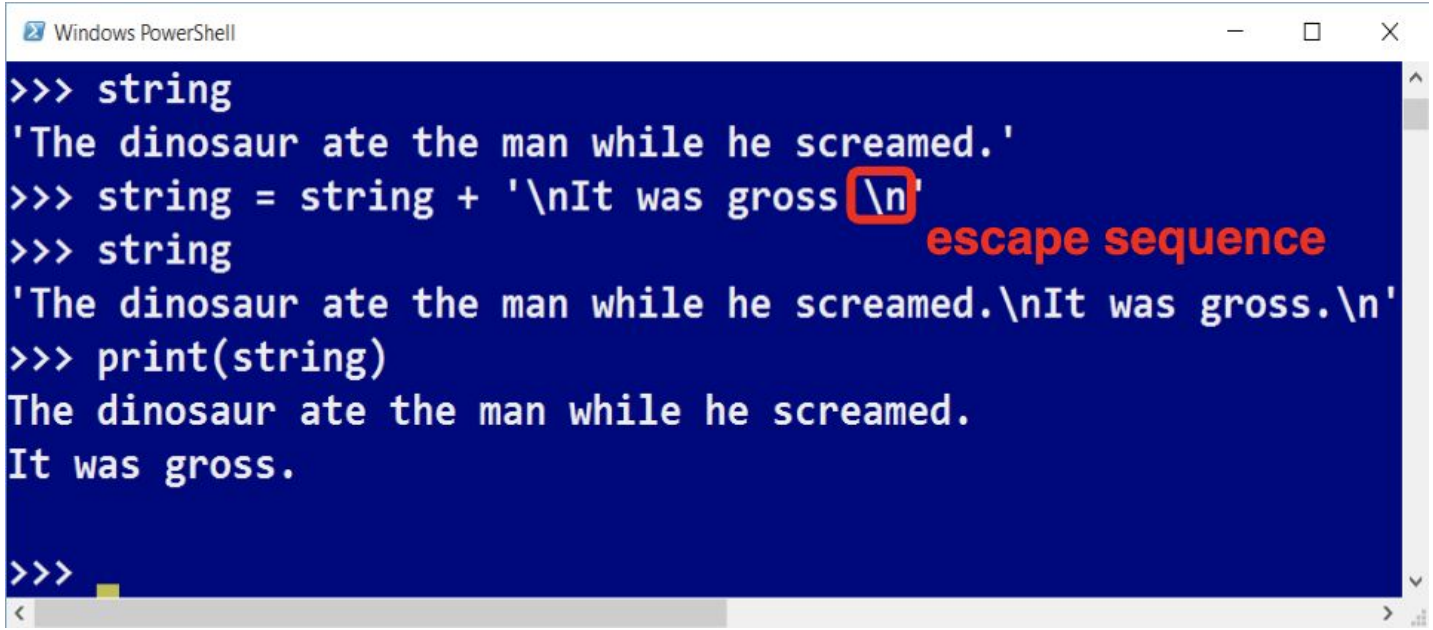


This is called operator overloading.

String Concatenation



String Concatenation



Escape sequences are treated as single characters when printing.

String Concatenation

